MECHATRONIC SYSTEM INTEGRATION
(MCTA 3203)

SEMESTER 1, 2025/2026

EXPERIMENT 6 :

PLC interfacing with microcontroller and PC over Ethernet/IP (ver 2.0)

SECTION 2

GROUP 12

NAME	METRIC NO.
AHMAD FARHAN BIN AHMAD NAHRUDI	2317373
MUHAMMAD AIMAN BIN SAFAWI	2318249
MOHD NAFIZ ZUHAIREE BIN MOHD NEEZAR	2317423

DATE OF SUBMISSION :
26/11/2025

ABSTRACT

This experiment explored how a PLC system can be programmed and controlled using OpenPLC software together with an Arduino board. The main goal was to simulate real industrial control logic and then apply it to physical hardware. Using the OpenPLC Editor, ladder diagrams were created, tested, and uploaded to the Arduino through USB communication. Two simple automation projects were carried out: a blinking LED using timer logic, and a Start–Stop control circuit using a latching system similar to those used in industry. During the process, variables were created and mapped to the correct Arduino pins, the hardware circuit was built using an LED, resistors, and push buttons, and the final ladder logic was tested on both the software simulator and the real hardware. Once uploaded, both circuits worked as expected—the LED blinked based on the set time interval, while the Start–Stop circuit responded correctly to button inputs. Overall, this experiment demonstrated how an Arduino can act as a low-cost PLC alternative for learning and testing automation concepts. It also provided hands-on experience with ladder logic programming, hardware interfacing, and system troubleshooting, which are essential skills in industrial automation and control engineering.

TABLES OF CONTENTS

ABSTRACT..... 2
TABLES OF CONTENTS.....3
1.0 INTRODUCTION.....4
2.0 MATERIAL AND EQUIPMENT.....5
3.0 EXPERIMENTAL SETUP.....6
4.0 METHODOLOGY.....7
4.1 Software Preparation..... 7
4.2 Ladder Diagram Creation.....7
4.3 Physical Address Mapping.....7
4.4 Uploading Ladder Logic to the Arduino..... 7
4.5 Hardware Setup..... 8
4.6 Testing and Verification.....8
5.0 DATA COLLECTION..... 9
6.0 DATA ANALYSIS..... 10
6.1 LED Blinking Circuit (Timer-Based Analysis).....10
6.2 Start–Stop Control Circuit (Latching Logic Analysis)..... 10
7.0 RESULTS..... 11
8.0 DISCUSSIONS..... 12
9.0 CONCLUSION.....13
10.0 RECOMMENDATION.....14
REFERENCES..... 15
APPENDICES..... 16
Appendix A..... 16
ACKNOWLEDGEMENT..... 18
STUDENTS DECLARATION..... 18
Certificate of Originality and Authenticity.....19

1.0 INTRODUCTION

Programmable Logic Controllers (PLCs) play an important role in modern industrial automation, where reliable, repeatable, and programmable control is required to operate machinery and manufacturing systems. Unlike general-purpose computers, PLCs are designed specifically to withstand harsh environments while continuously monitoring inputs and controlling outputs in real time. Because of their importance in industry, understanding how PLCs are programmed, interfaced, and applied in real hardware systems is a fundamental skill in automation and control engineering.

In this experiment, the focus was to explore PLC operation using OpenPLC software together with an Arduino microcontroller acting as the hardware execution platform. OpenPLC allows users to design ladder logic programs—one of the most widely used programming languages in industrial automation—due to its similarity to traditional relay-based circuits and its intuitive visual structure. By combining software simulation with real hardware execution, this experiment bridges theoretical concepts with hands-on practical experience.

Two ladder logic programs were developed and tested: a timer-based blinking LED circuit and a Start–Stop latching control circuit. The blinking LED program demonstrates the use of timing functions in automation, where a delay or periodic signal is required. The Start–Stop circuit represents a very common industrial control application, such as motor control, where a latch logic holds the output ON until a Stop command is triggered.

Throughout the experiment, students were required to define variables, map them to Arduino physical input/output pins, configure communication settings, upload the logic to the board, and finally test the circuit using breadboard components. This workflow closely reflects real PLC programming steps: design, compile, download, and run.

By successfully completing this setup, the experiment introduces core automation concepts such as logic execution cycles, variable addressing, hardware interfacing, and ladder logic functionality. More importantly, it demonstrates how low-cost platforms like Arduino can be used as educational substitutes for industrial PLCs, making automation learning more accessible while still maintaining relevance to real-world applications.

2.0 MATERIAL AND EQUIPMENT

The following materials and equipment were used during this laboratory session, consistent with the experiment documentation provided:

- OpenPLC Editor software
- Arduino Board
- 2 Push Button Switches
- LED
- Resistors
- Jumper Wires
- Breadboard
- USB Cable

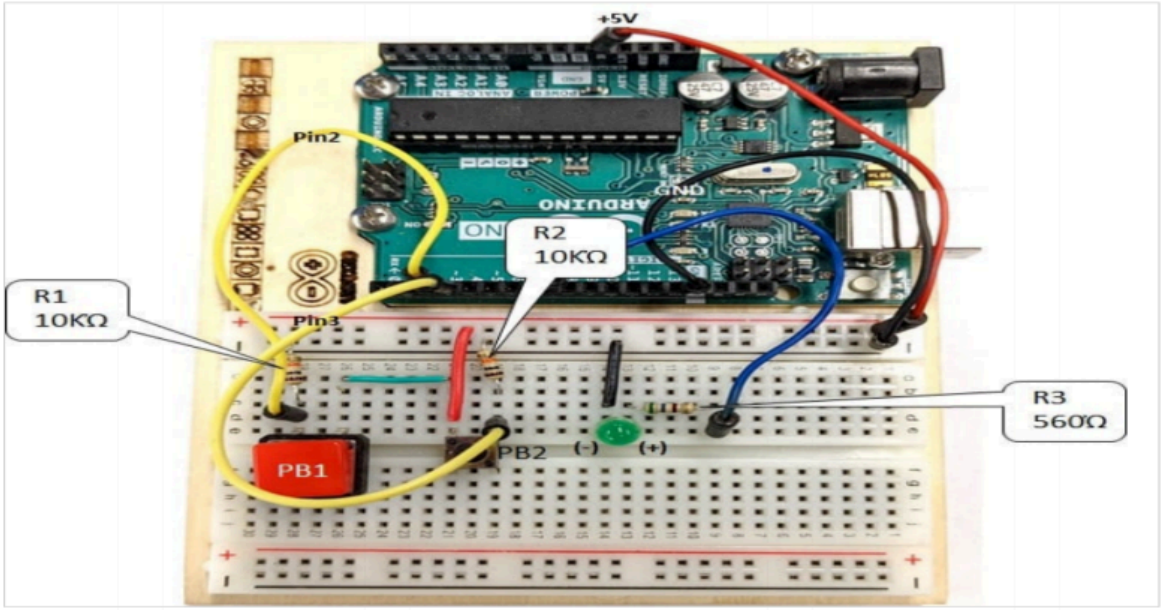

Figure 4. *The Arduino Uno PLC electronic circuit schematic diagram.*

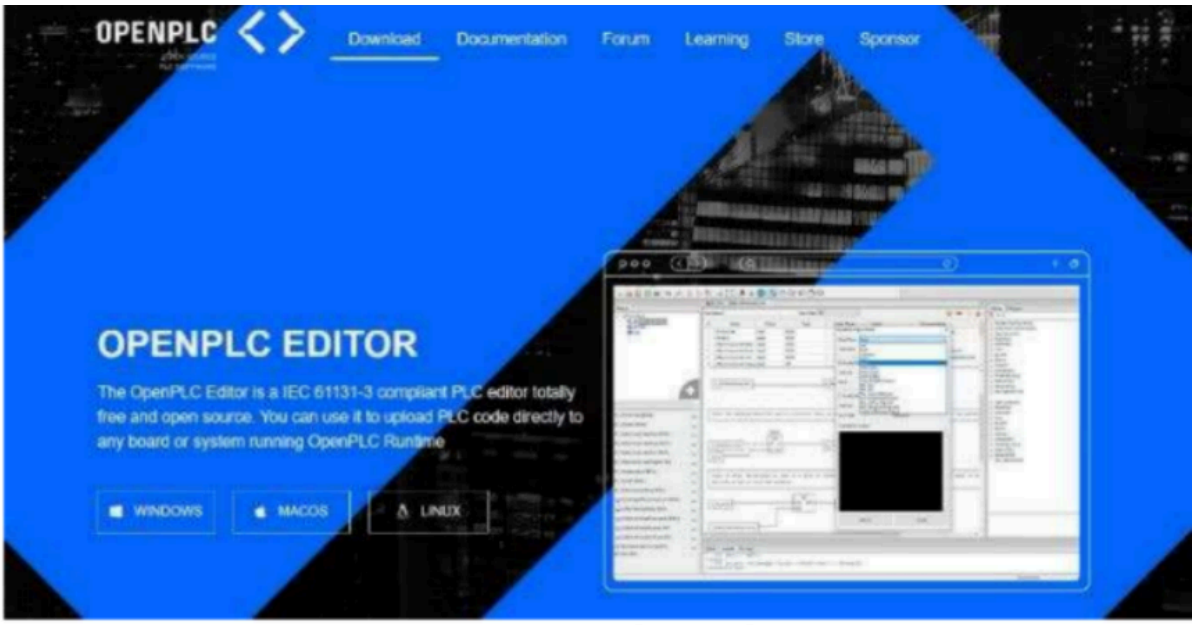

Fig.1: OpenPLC Editor website

3.0 EXPERIMENTAL SETUP

The experimental setup is built around three key components. First, the OpenPLC Editor running on a PC is used to create the ladder logic program, define variables, configure I/O addressing, and finally upload the compiled PLC code to the Arduino. Second, the Arduino board serves as the PLC runtime platform, executing the uploaded ladder logic by interacting with its digital input and output pins. Finally, the hardware circuit consists of the physical input and output devices, including the LED, START and STOP push buttons, resistors, jumper wires, and breadboard connections.

This setup was used to perform two main tasks:

- A timer-based LED blinking program, where the LED toggles ON and OFF according to a programmed time delay.

- A Start–Stop control circuit that mimics real motor control logic, using a latching method controlled by push buttons and a single LED indicator.

4.0 METHODOLOGY

4.1 Software Preparation

1. Install the OpenPLC Editor on the computer.

2. Open the software and set up a new project directory.

3. Create a new Program Organization Unit (POU) and select Ladder Diagram (LD) as the programming format.

4.2 Ladder Diagram Development

1. Define all required variables (Class: Local, Data Type: BOOL).

2. Insert the left and right power rails into the workspace.

3. Add ladder components such as NO/NC contacts, coils, and timer blocks if needed.

4. Design the blinking LED ladder logic based on the reference diagram.

5. Compile the program to ensure there are no errors and run a simulation.

4.3 Assigning Physical Addresses

1. Determine which Arduino pin will be used as the output.

2. Check the OpenPLC documentation to find the corresponding digital I/O address (e.g., Arduino Mega pin 14 → %QX0.0).

3. Enter this address into the variable configuration in the ladder logic.

4.4 Uploading the Logic to Arduino

1. Connect the Arduino board to the PC using a USB cable.

2. Identify the correct COM port assigned to the board.

3. Click the upload/transfer button in OpenPLC.

4. Choose the appropriate Arduino model from the list.

5. Input the COM port and complete the upload process.

4.5 Hardware Assembly

Components used in the Start–Stop circuit include:

- START push button

- STOP push button

- LED (output indicator)

- Required resistors for pull-up or pull-down configuration

- Breadboard and jumper wires

Connections were made according to the schematic provided to ensure proper I/O behavior.

4.6 Testing and Validation

1. Power up the Arduino and allow it to execute the uploaded ladder program.

2. Verify that the LED blinks as programmed for the first task.

3. Test the Start–Stop control:

    - Press START → LED should activate and stay ON (latched).

    - Press STOP → LED should turn OFF.

4. Note observations, troubleshoot wiring or logic if operation is incorrect.

8.0 DISCUSSION

This experiment provided a practical understanding of how PLC systems can be implemented and tested using OpenPLC software and an Arduino microcontroller. Although PLCs are typically industrial-grade devices designed for automation tasks, using an Arduino as the execution platform allowed the core concepts of PLC programming such as input and output mapping, ladder logic execution, and latching behavior to be explored in a more accessible and cost-effective way.

One of the key observations from the experiment is how the OpenPLC runtime on the Arduino mimics the scan cycle behavior of a real PLC. In a typical PLC, the system continuously performs a loop where inputs are read, logic is executed, and outputs are updated. This same operational cycle could be observed in the uploaded ladder logic

programs. For example, in the LED blinking task, changes in the timer block were reflected during runtime, demonstrating how time-based control works within a PLC environment.

The Start–Stop control circuit further demonstrated an important industrial concept: latching. When the START push button was pressed, the LED remained ON even after the button was released. This behavior simulated how motors or machinery remain operating until a STOP command is given. This type of circuit is widely used in automation, especially in conveyor systems, pumps, and motor-driven devices. The proper execution of the latch logic showed that the software and hardware integration was successful and that the program behaved similarly to real-world PLC systems.

Another important aspect of the experiment was the process of physical address mapping. The need to match variables in the ladder logic with Arduino physical pins helped reinforce the relationship between software instructions and hardware execution. Any mismatch in addressing or wiring would result in unexpected behavior, and this process helped develop troubleshooting skills. In some cases, errors such as incorrect COM port selection, wrong pin assignment, loose jumper wires, or incorrect resistor configurations could cause the circuit not to respond as expected. Identifying and correcting these issues was an integral part of completing the experiment successfully.

Overall, the system performed reliably after configuration, and both ladder logic programs functioned as intended. The experiment demonstrated that OpenPLC, paired with an Arduino, can serve as an effective learning platform to practice PLC programming without requiring expensive industrial hardware. This approach is especially beneficial for education environments where scalability and affordability are important.

From this activity, it is clear that the experience gained goes beyond just software programming it includes understanding of control system behavior, circuit wiring, digital I/O interaction, and systematic debugging. These skills are directly applicable

to industrial automation environments, making this experiment a meaningful introduction to PLC-based control systems.

5.0 DATA COLLECTION

The data collected from the PLC–Arduino interfacing experiment shows that the ladder diagrams created in OpenPLC compiled successfully, simulated correctly, and produced the expected blinking LED behavior. After assigning the correct physical addressing (such as %QX0.0) and uploading the program to the Arduino, the LED output matched the simulated operation. The timer-based modification also produced noticeable changes in blinking intervals both in simulation and on hardware. For the Start–Stop control circuit, the LED responded consistently to the START and STOP inputs, demonstrating proper latching and control behavior as described in the task instructions.

6.0 DATA ANALYSIS

The collected data indicates that all PLC logic functions blinking, timing, and Start Stop latching performed exactly as intended, confirming the correct implementation of ladder logic, variable assignment, and microcontroller interfacing. The alignment between simulation results and actual hardware output demonstrates a solid understanding of PLC concepts such as contact–coil operation, TON timing, and latch circuits. The consistency of LED responses verifies that both the logic design and physical circuit wiring were accurately executed, fulfilling the objectives of demonstrating basic PLC control using OpenPLC and Arduino.

7.0 RESULTS

The results of the experiment show that the programmed ladder logic tallied perfectly with the behavior of the PLC runtime on the Arduino. The LED blinked according to the ladder diagram, the timer block produced the expected delay changes, and the Start–Stop circuit operated accurately with proper latching action. Every output observed on the Arduino matched the simulation results in OpenPLC, confirming that the logic, pin mapping, and execution were fully consistent with the intended PLC control operation

9.0 CONCLUSION

In conclusion, this experiment successfully demonstrated how PLC-style automation can be programmed, simulated, and executed using the OpenPLC platform together with an Arduino microcontroller. Through the hands-on process of setting up ladder logic, assigning variables, mapping hardware addresses, and testing real input and output devices, the experiment provided a realistic introduction to industrial PLC concepts without requiring fully specialized industrial hardware.

Both programmed tasks—the timer-controlled blinking LED and the Start–Stop latching circuit—operated as expected once the hardware and software setups were completed. The blinking LED exercise helped reinforce the concept of timer operations and how PLCs manage time-dependent processes. Meanwhile, the Start–Stop circuit demonstrated one of the most fundamental automation principles: maintaining system output through latching logic until a stop command is triggered. This behavior closely resembles how motors, conveyors, pumps, and other industrial equipment operate in real-world environments.

Throughout the experiment, several important skills were developed, including ladder logic programming, physical address configuration, hardware connection planning, and debugging. These are essential competencies for anyone pursuing work in automation, industrial systems, or embedded control design. The experience also highlighted the importance of accuracy in variable addressing and wiring—small mistakes such as selecting the wrong COM port or misplacing a jumper wire could prevent the system from functioning correctly.

Overall, the experiment demonstrated that an Arduino can act as a practical and affordable PLC substitute for learning purposes, making automation concepts more accessible to students and beginners. Beyond simply executing logic, the result reinforced a deeper understanding of how software and hardware work together to produce automated control behavior. With this foundation, the system can be expanded in future work to include sensors, relays, motors, HMI screens, and communication networking, moving closer to real industrial automation systems.

10.0 RECOMMENDATION

Based on the outcomes of this experiment, several improvements and extensions can be suggested to enhance learning effectiveness and system capability. First, it is recommended to continue exploring additional ladder logic functions such as counters, comparators, analog input handling, and sequential control. These functions represent common operations in industrial automation and would provide a deeper understanding of real PLC functionality beyond simple timers and latching logic.

Another recommendation is to integrate more advanced hardware components. Instead of only controlling an LED, future experiments could include devices such as DC motors, relays, buzzers, or solenoid valves. Incorporating these elements would allow students to observe how PLC logic influences mechanical systems, making the experiment more realistic and engaging. Likewise, adding input sensors such as limit switches, proximity sensors, or temperature sensors would help demonstrate how automation systems react dynamically to environmental changes.

It is also suggested to expand the communication aspect of the experiment. Currently, the Arduino is programmed locally, but future activities could include network-based control using protocols such as Modbus TCP, MQTT, or Ethernet/IP. This would help simulate factory-level communication and supervisory control systems, which are commonly seen in SCADA or Industry 4.0 environments.

Additionally, improving documentation and system labeling would help reduce wiring errors and save troubleshooting time. Using proper cable management, pin labeling, and circuit diagrams can make the setup cleaner and easier for future modification.

Students may also benefit from maintaining logs of their test results, wiring issues, or error messages, as this helps improve diagnostic skills—an essential trait for automation engineers.

Finally, adopting a structured safety practice is recommended, especially when moving toward higher-voltage or motor-driven systems. Although this experiment operated at low and safe voltages, applying safety concepts early prepares students to work confidently with industrial PLCs and electrical systems.

Overall, these recommendations aim to support continued improvement of the learning process and encourage students to develop more advanced and practical automation projects in the future.

11.0 REFERENCE

1. Wilcher, D. (2022, August 30). *PLC ladder logic on an Arduino: Introduction to OpenPLC*. Control. Retrieved from https://control.com/technical-articles/plc-ladder-logic-on-an-arduino-introduction-to-openplc/

2. Autonomy – open-source PLC software. (n.d.). https://autonomylogic.com/

3. Wilcher, D. (2022, September 28). PLC Ladder Logic on an Arduino: Building a Start-Stop Circuit. Control.com. https://control.com/technical-articles/plc-ladder-logic-on-an-arduino-building-a-start-stopcircuit/ (control.com)

4. Mechatronics System Integration Lab Module - Google Drive https://drive.google.com/drive/folders/1QecGGN96D76UL2gwBjevFwiEXv2UEniO

9.0 ACKNOWLEDGEMENT

We would like to sincerely thank the instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal Abidin and the lab technicians, for all of their help,encouragement and support during this project. Their knowledge and perceptions have greatly influenced the course of this work. Additionally, we would like to express our gratitude to our peers for their support and cooperation, both of which were crucial to the accomplishment of this project.

10.0 STUDENT'S DECLARATION

We hereby declare that, except for the places where it is acknowledged, all of the work presented in this report is entirely ours. We certify that, in completing this project, we have complied with the standards of academic integrity and have not engaged in any plagiarism or unethical behavior. Every information and support source used in this work has been appropriately referenced and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature: ***ahmadfarhan***

Name: Ahmad Farhan bin Ahmad Nahrudi
Matric Card: 2317373

Read[/]
Understand[/]
Agree[/]

Signature: ***aimansafawi***

Name: Muhammad Aiman bin Safawi
Matric Card: 2318249

Read[/]
Understand[/]
Agree[/]

Signature: ***nafizzuhairee***

Name: Mohd Nafiz Zuhairee bin Mohd Neezar
Matric Card: 2317423

Read[/]
Understand[/]
Agree[/]